Vulnerability Assessment Report

Target Application: ShopNow (E-Commerce Web App)

Prepared by: Poli Reddy Dondapati

Date: August 2026

Target(s): http://localhost:3000 (React front-end), http://localhost:5000 (API)

Assessment Type: Passive / read-only web application scan

Tools Used: OWASP ZAP (Passive Scan), Nmap, Browser DevTools

Methodology: OWASP-aligned passive reconnaissance

1. Executive Summary

A passive security assessment was performed against the ShopNow demo web application to identify common web-application misconfigurations and information-disclosure issues. The engagement was strictly passive and non-intrusive — no exploitation, brute forcing, login bypass, or denial-of-service activity was attempted, in line with the scope and ethics defined for this exercise.

The scan identified 12 findings across the application, summarized by severity below:

Severity	Count
High	0
Medium	3
Low	4
Informational	5

No critical or high-risk vulnerabilities were identified. The findings are primarily missing security headers and minor information disclosure issues — common in default framework configurations (Express.js / Create React App) and typically low-effort to remediate.

2. Target Overview

The primary target of this assessment was the ShopNow demo e-commerce application, running at localhost:3000 (React front-end) with its API at localhost:5000. A screenshot of the application under test is included as evidence in Finding 6.1 below.

3. Scope & Ethics

In scope:

- Public-facing pages of the target application
- Passive scanning (OWASP ZAP Passive Scanner)
- HTTP security header review
- Client-side configuration analysis

Explicitly out of scope:

- Login bypass or credential attacks

- Exploitation of any kind
- Brute-force attacks
- Denial-of-Service (DoS) testing
- Any activity capable of degrading or damaging the target

This assessment follows a security-auditor approach rather than an attacker approach — the goal is to document and explain risk, not to demonstrate exploitation.

4. Methodology

1. Target selection – ShopNow demo application (localhost:3000 front-end, localhost:5000 API).
2. Read-only analysis – traffic was proxied through OWASP ZAP (localhost:8080) to passively observe requests/responses, headers, and client-side JavaScript without sending attack payloads.
3. Header & configuration review – response headers were checked against security best practice (CSP, HSTS, X-Frame-Options, X-Content-Type-Options, CORS policy).
4. Documentation – each finding was logged with URL, evidence, and ZAP's CWE/WASC classification.
5. Risk classification – findings rated Low / Medium / High based on exploitability and business impact.
6. Remediation planning – practical, business-friendly fixes were mapped to each finding.

5. Findings Summary

#	Finding	Risk	Confidence	Affected Location
1	Content Security Policy (CSP) Header Not Set	Medium	High	Site-wide (localhost:3000)
2	Cross-Domain Misconfiguration (permissive CORS)	Medium	Medium	localhost:3000/
3	Missing Anti-Clickjacking Header (X-Frame-Options)	Medium	—	localhost:3000/
4	Server Leaks Info via X-Powered-By Header	Low	Medium	localhost:3000/
5	Strict-Transport-Security (HSTS) Header Not Set	Low	—	Site-wide
6	Timestamp Disclosure (Unix epoch in JS bundle)	Low	Low	localhost:3000/ static/js/bundle.js
7	X-Content-Type-Options Header Missing	Low	—	8 endpoints
8	Authentication Request Identified	Info	High	localhost:5000/ api/Login
9	Information Disclosure – Suspicious Comments	Info	—	12 instances
10	Modern Web	Info	Medium	localhost:3000/

#	Finding	Risk	Confidence	Affected Location
	Application Detected			
11–12	Additional informational findings	Info	—	Various

6. Detailed Findings

6.1 Content Security Policy (CSP) Header Not Set — Medium

What it is: The application does not send a Content-Security-Policy header, so the browser has no restriction on which sources scripts, styles, or frames can be loaded from.

Why it matters: Without CSP, a successful cross-site scripting (XSS) bug elsewhere in the app becomes far more dangerous, since malicious scripts can execute and load external resources freely.

Evidence: Missing on all tested pages (CWE-693, WASC-15).

Remediation: Add a Content-Security-Policy header (e.g. default-src 'self') and tighten it iteratively as the application's real resource needs are mapped out.

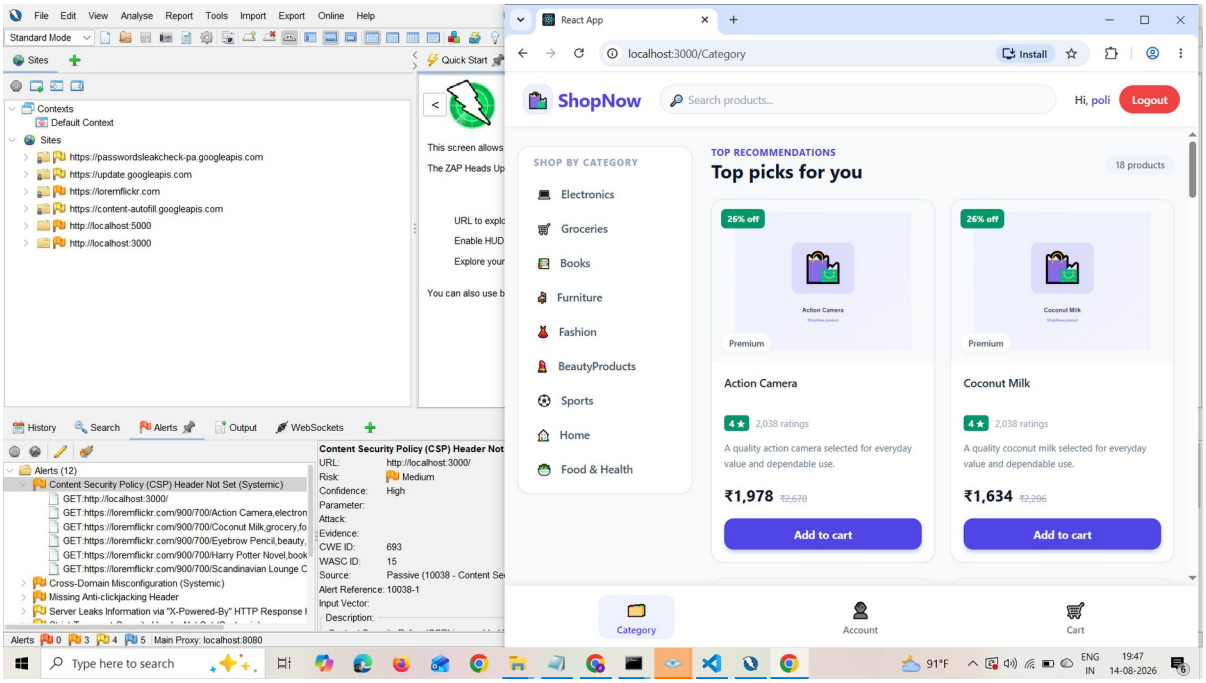

Figure 1: CSP Header Not Set – ZAP Alert (ShopNow storefront under test)

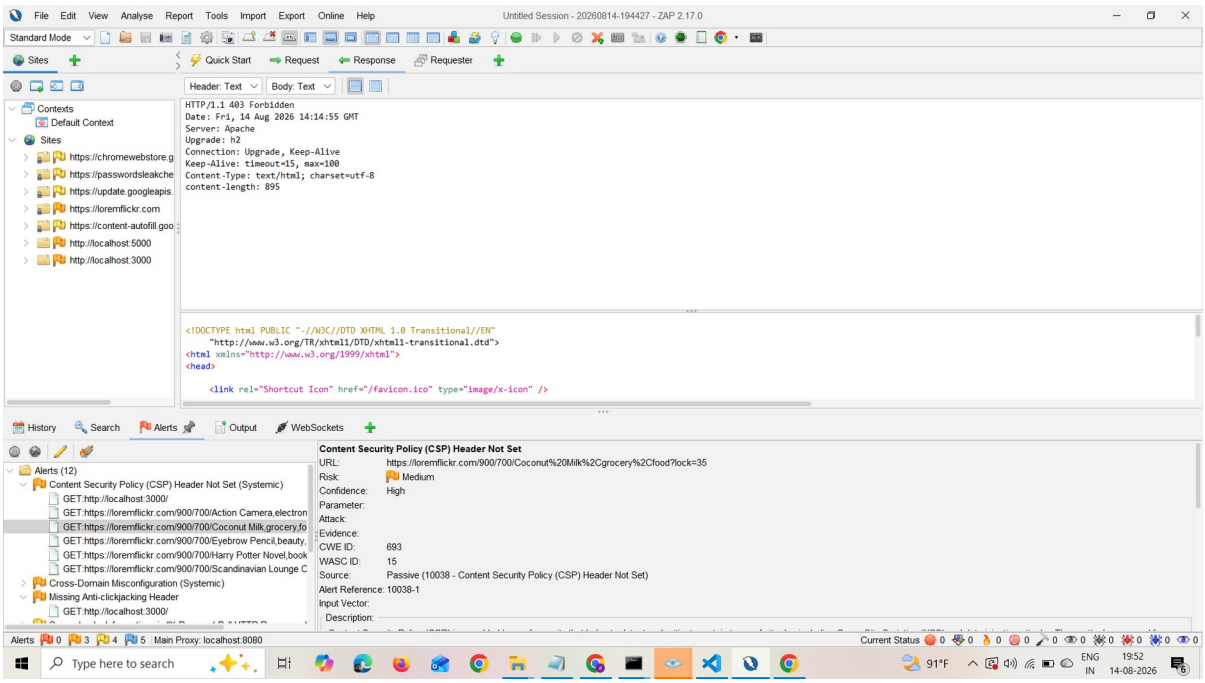

Figure 2: CSP Header Not Set – Request/Response Detail

6.2 Cross-Domain Misconfiguration (CORS) — Medium

What it is: The API responds with Access-Control-Allow-Origin: *, Access-Control-Allow-Methods: *, and Access-Control-Allow-Headers: *, allowing any website to make cross-origin requests to it.

Why it matters: Any third-party site could call this API from a user's browser and potentially read responses, increasing the risk of data leakage if the API ever returns sensitive data.

Evidence: Access-Control-Allow-Origin: * (CWE-264, WASC-14).

Remediation: Restrict CORS to a known, trusted list of origins instead of a wildcard, especially for any endpoint that handles user or account data.

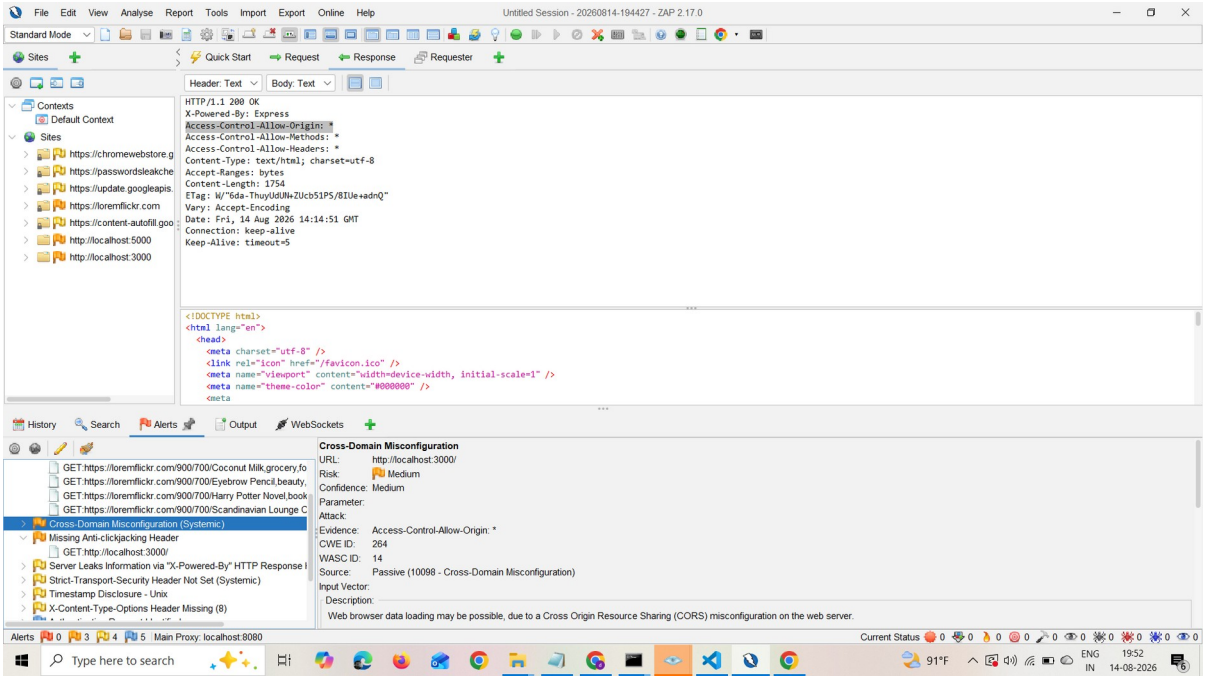

Figure 3: Cross-Domain Misconfiguration – ZAP Alert

6.3 Missing Anti-Clickjacking Header — Medium

What it is: No X-Frame-Options (or CSP frame-ancestors) header is present, so the page can be loaded inside an <iframe> on another site.

Why it matters: This enables clickjacking attacks, where a malicious site overlays invisible UI on top of the real application to trick users into clicking buttons (e.g. "Add to cart", "Logout") without realizing it.

Remediation: Add X-Frame-Options: DENY (or SAMEORIGIN) or a CSP frame-ancestors 'none' directive.

6.4 Server Leaks Information via X-Powered-By Header — Low

What it is: The server responds with X-Powered-By: Express, revealing the backend framework.

Why it matters: This makes it easier for an attacker to look up known vulnerabilities for that specific framework/version.

Evidence: X-Powered-By: Express (CWE-497, WASC-13).

Remediation: Disable the header, e.g. app.disable('x-powered-by') in Express.

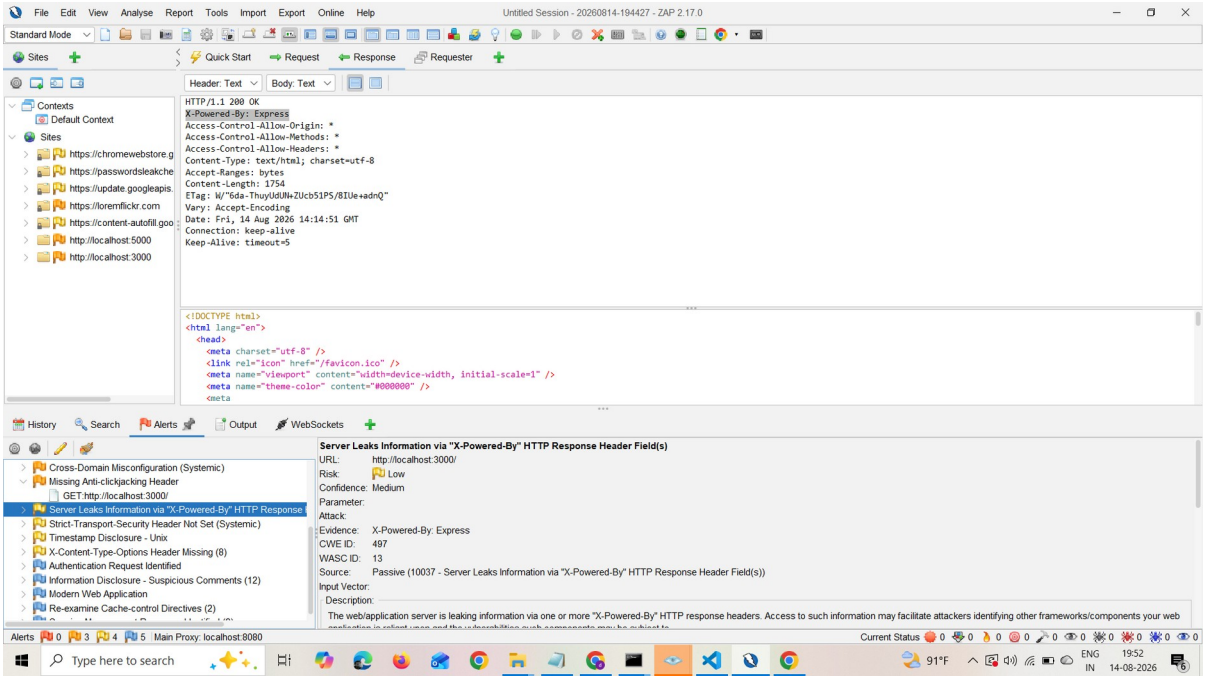

Figure 4: X-Powered-By Header Leak – ZAP Alert

6.5 Strict-Transport-Security (HSTS) Header Not Set — Low

What it is: The application does not instruct browsers to always use HTTPS for future visits.

Why it matters: Without HSTS, users are more exposed to protocol-downgrade and man-in-the-middle attacks on untrusted networks.

Remediation: Once the site is served over HTTPS in production, add Strict-Transport-Security: max-age=31536000; includeSubDomains.

6.6 Timestamp Disclosure — Low

What it is: A Unix timestamp value was found embedded in the client-side JavaScript bundle (bundle.js).

Why it matters: On its own this is low-risk, but embedded timestamps can sometimes hint at build/deploy schedules or internal versioning.

Evidence: Raw value 2080374784 found in bundle.js (CWE-497, WASC-13).

Remediation: Not urgent; consider excluding unnecessary debug/build metadata from production bundles.

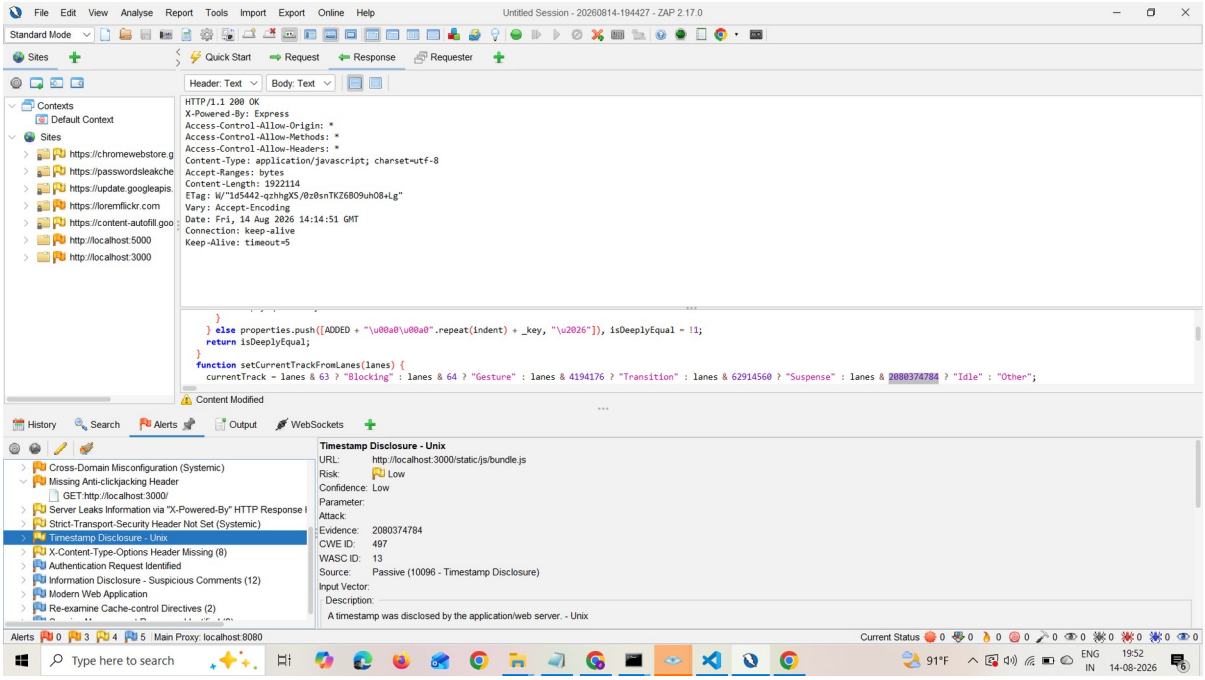

Figure 5: Timestamp Disclosure – ZAP Alert

6.7 X-Content-Type-Options Header Missing — Low

What it is: The X-Content-Type-Options: nosniff header is missing across 8 endpoints, allowing browsers to "MIME-sniff" content types.

Why it matters: This can allow browsers to misinterpret files (e.g. treating a file as executable script when it shouldn't be), slightly increasing XSS risk.

Remediation: Add X-Content-Type-Options: nosniff globally at the web server or application middleware layer.

6.8 Authentication Request Identified — Informational

What it is: ZAP passively identified a login request (POST /api/Login) containing an email and password field.

Why it matters: This is expected behavior for a login flow and is not a vulnerability by itself — flagged only so it can be confirmed the endpoint is served over HTTPS and rate-limited in production.

Remediation: Ensure the login endpoint is only ever served over HTTPS, and confirm rate-limiting/lockout controls exist against credential stuffing (outside the scope of this passive assessment to verify).

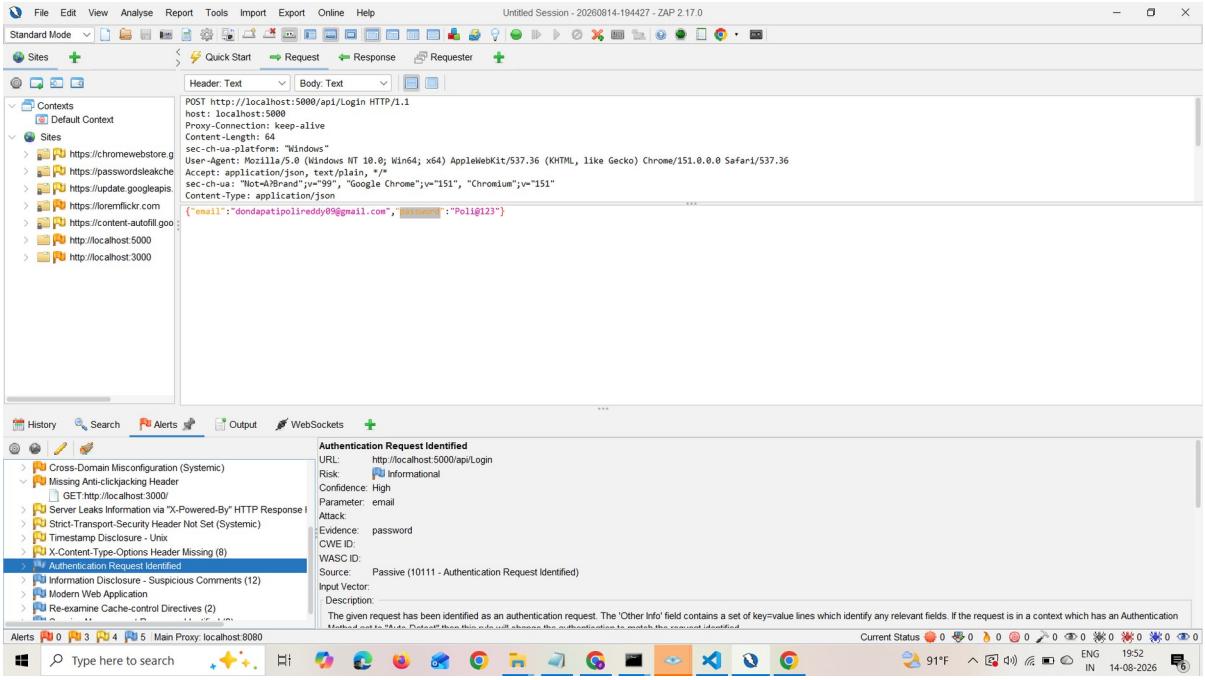

Figure 6: Authentication Request Identified – ZAP Alert

6.9 Information Disclosure – Suspicious Comments — Informational

What it is: ZAP flagged 12 instances of comments in application source/JS that may contain developer notes.

Why it matters: Leftover comments can occasionally reveal internal logic, TODOs, or debug information not meant for public release.

Remediation: Review and strip non-essential comments from production builds.

6.10 Modern Web Application Detected — Informational

What it is: ZAP identified the app as a modern JavaScript single-page application (React, served via bundle.js).

Why it matters: No risk — informational only. Suggests using ZAP's Client Spider/AJAX Spider for more thorough coverage in future scans.

Remediation: None required.

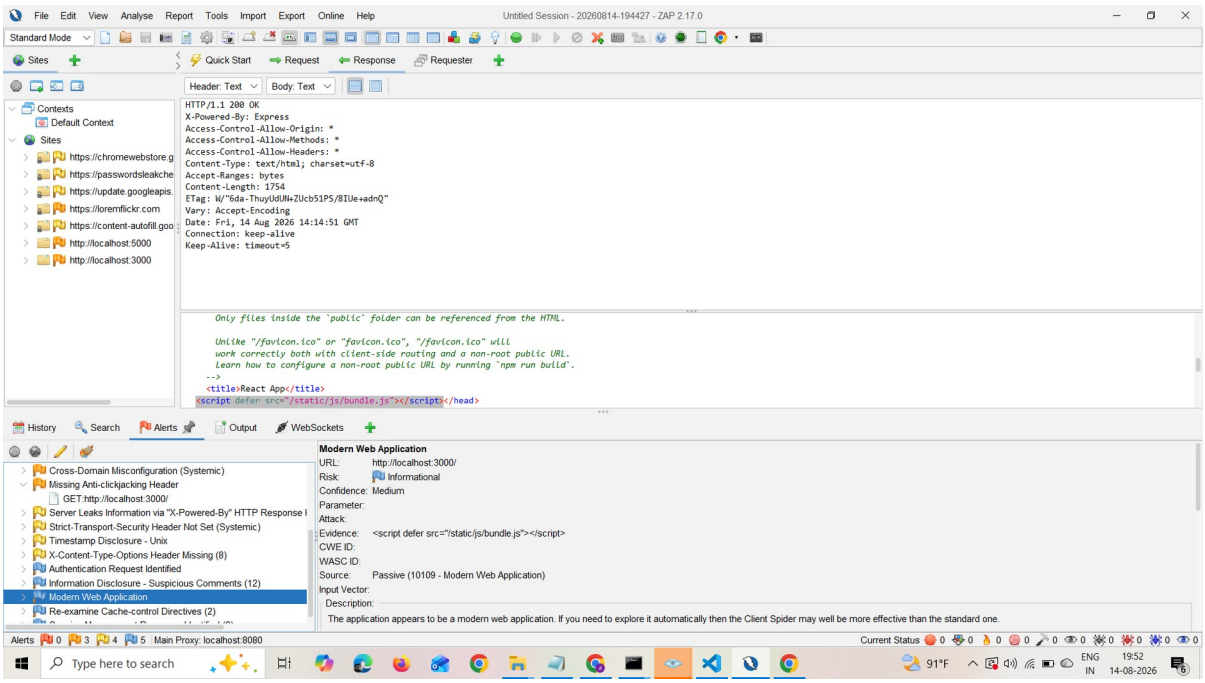

Figure 7: Modern Web Application Detected – ZAP Alert

7. Risk Classification Reference

Level	Meaning
High	Directly exploitable issue with significant business impact — none found
Medium	Weakens the application's defenses and should be fixed in the near term
Low	Minor hardening opportunity, low likelihood of direct exploitation
Informational	No action required; useful context or expected behavior

8. Recommendations (Priority Order)

1. Add missing security headers — CSP, X-Frame-Options, X-Content-Type-Options, and HSTS. This single change addresses 6 of the 12 findings.
2. Lock down CORS policy — replace wildcard origins with an explicit allow-list.
3. Remove framework fingerprinting — disable the X-Powered-By header.
4. Clean up production builds — strip developer comments and unnecessary metadata from shipped JavaScript.
5. Re-scan after fixes — confirm all header-based findings are resolved and re-run the passive scan.

9. Conclusion

The ShopNow application shows no evidence of high-risk or actively exploitable vulnerabilities from this passive assessment. The findings are consistent with a default Express/React setup that has not yet had production security hardening applied. Implementing standard HTTP security headers and tightening the CORS policy would resolve the large majority of issues identified and meaningfully improve the application's security posture.

This report was produced through passive, non-destructive scanning only, in accordance with the ethical scope defined for this assessment. No exploitation, authentication bypass, or service disruption was performed or attempted.